SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Ranjitha. R	**Reg. No.:**	192421285
Date:	08.10.2026	**Duration:**	60 minutes

Code of Conduct

I Ranjitha (192421285) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Annexure A – Case Study Analysis

Machine Learning Model Drift

Scenario

A fraud detection system at a payment processor was highly accurate at launch but its precision has degraded significantly over 18 months as fraudsters adapted their techniques.

Questions

a) How would you design an MLOps pipeline to detect and address model drift continuously?

b) Discuss monitoring strategies, retraining triggers, shadow deployment, and A/B testing of models.

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
   - Justify why the selected methodology/framework is suitable for the proposed system.
   - Explain how it supports successful project development and maintenance.
6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software	Proposes an innovative, feasible solution applying	Suitable solution with good application of software	Basic solution with limited application of software	Methodology is inappropriate, poorly justified, or not discussed.

Engineering Principles (15%)	appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	engineering principles.	engineering principles.	
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Case Study Scenario

A payment processor deployed a machine learning model to detect fraudulent transactions. At launch, the model achieved high precision. Over 18 months, its precision degraded significantly because fraudsters adapted their techniques. The assessment asks how an MLOps pipeline can detect and address model drift continuously and how monitoring, retraining triggers, shadow deployment, and A/B testing can be used.

Report Structure

This report follows the assessment requirements: problem understanding; stakeholder and challenge analysis; current-system analysis; a proposed software solution; software engineering principles; methodology justification; expected outcomes, limitations, and future enhancements. Diagrams are explanatory design aids. Any sample trend shown is illustrative because the supplied case study does not provide monthly measurements.

1. Understand the Problem Statement

1.1 Problem Description

Machine learning models learn patterns from historical examples. In a payment fraud detection system, patterns may involve transaction amount, frequency, merchant category, device signals, location, and other permitted features. Fraudsters continually change their behaviour to bypass detection. Over time, the transactions encountered in production may differ from the data used to train the model, or the relationship between transaction characteristics and fraud outcomes may change.

Model drift is a general term for changes that can make a deployed model less suitable for current conditions. Data drift occurs when input-feature distributions change. Concept drift occurs when the relationship between inputs and the target outcome changes. A shift in prediction scores or flag rates can be an early warning, but it does not by itself prove that real-world precision has declined. Performance evaluation requires reliable labels.

1.2 Objectives

- Continuously monitor incoming transaction data and detect meaningful changes.
- Measure performance with precision, recall, false-positive rate, and appropriate complementary metrics.

Edit with WPS Office

- Define clear alerts and retraining triggers based on sustained evidence and business risk.
- Evaluate candidate models before they influence real customer decisions.
- Maintain traceability for datasets, code, model versions, approvals, and deployments.
- Reduce avoidable fraud losses while limiting incorrect declines of legitimate transactions.

1.3 Scope

The proposed solution covers data ingestion, data validation, feature and prediction monitoring, delayed-label performance monitoring, alert generation, retraining, model evaluation, version management, shadow deployment, A/B testing, and rollback. It is designed to support fraud analysts, data scientists, platform engineers, and business risk teams. It does not guarantee that every emerging fraud tactic will be detected immediately; human investigation, business rules, and operational controls remain necessary.

1.4 Why the Problem Matters

A decline in precision means that a larger share of transactions flagged as fraudulent may be legitimate. Customers may experience payment declines, merchants may lose sales, and analysts may spend more time investigating false alarms. If recall also declines, fraudulent transactions may pass through undetected and cause financial losses. A reliable solution therefore needs to balance fraud prevention, customer experience, speed, and operational cost.

1.5 Performance Measures and Terminology

Measure	Definition / purpose	Interpretation
Precision	True positives divided by all transactions flagged positive.	Low precision can mean more false alarms.
Recall	True positives divided by all actual positives.	Low recall means more fraud may be missed.
False-positive rate	Legitimate transactions incorrectly flagged divided by all legitimate transactions.	Helps estimate unnecessary friction.
PR-AUC	Area under the precision–recall curve.	Useful for evaluating imbalanced classification.
Latency	Time taken to return a transaction score.	Important for payment-service responsiveness.

Edit with WPS Office

Drift indicator	Statistical measure of changes in data or prediction patterns.	A signal to investigate, not definitive proof of failure.

Precision = TP / (TP + FP), where TP is the number of correctly detected fraudulent transactions and FP is the number of legitimate transactions incorrectly flagged. Recall = TP / (TP + FN), where FN represents fraudulent transactions that were not detected.

1.6 Key Assumptions

- Confirmed transaction outcomes become available eventually, although they may be delayed.
- The processor has permission to use selected transaction features for fraud prevention and model monitoring.
- Business owners can define acceptable risk thresholds and approve production releases.
- The proposed system can store model versions and relevant monitoring records securely.
- The exact existing architecture is not specified in the case study; this report proposes a suitable design rather than claiming that it already exists.

2. Identify Key Stakeholders and Challenges

Stakeholder	Role and responsibility
Fraud analysts	Investigate alerts, confirm transaction outcomes, identify new tactics, and provide feedback.
Data scientists / ML engineers	Design features, measure drift, retrain models, and evaluate candidates.
MLOps / platform engineers	Automate data and model pipelines, releases, logging, monitoring, and rollback.
Business and risk managers	Define acceptable fraud risk, approve thresholds, and balance losses against customer friction.
Customers and merchants	Use the payment service and report suspicious transactions or incorrect declines.
Security, privacy, and compliance teams	Protect transaction data, review access, and maintain appropriate audit records.

Edit with WPS Office

2.1 Stakeholder Challenges

Fraud Analysts

Analysts face large volumes of alerts and must distinguish real fraud from legitimate activity. New fraud techniques may not match known investigation patterns. Feedback may also be delayed because some transactions are confirmed only after customer reports, chargebacks, or additional checks. The system should provide understandable alert reasons, model version information, and a simple way to record investigation outcomes.

Data Scientists and ML Engineers

The modelling team must determine whether a change is caused by drift, poor data quality, seasonal behaviour, or an actual change in fraud tactics. Label delay and class imbalance can make performance estimates unstable. Candidate models must be evaluated on data that reflects the time order of real transactions so that future information does not leak into training.

Platform and Business Teams

Platform engineers must keep scoring services available and fast while maintaining reproducible deployments. Business teams need measurable criteria to decide whether a new model is better overall, not just on one metric. A model that improves recall but creates many false declines may not be acceptable without a revised threshold or additional controls.

2.2 Major Technical and Operational Challenges

- Concept drift: the mapping between transaction patterns and fraud outcomes changes.
- Data drift: input values and category frequencies move away from the training baseline.
- Delayed ground truth: reliable labels may arrive after a long investigation period.
- Class imbalance: high overall accuracy can hide poor detection of the minority fraud class.
- False positives: overly aggressive decisions can damage customer trust and merchant revenue.
- Privacy and security: transaction information requires restricted access and careful retention.
- Alert fatigue: too many low-value warnings can cause important alerts to be overlooked.
- Release risk: an untested model can perform worse than the stable production version.

2.3 Stakeholder Collaboration

The solution requires shared ownership. Analysts explain new fraud patterns and validate labels; data scientists investigate metrics and train candidates; platform engineers maintain reliable infrastructure; business owners define risk limits; and security teams review access and data

Edit with WPS Office

protection. A recurring model-review meeting can examine alerts, model performance, unresolved incidents, and proposed releases.

3. Analyze the Current Approach

3.1 Existing Process

The supplied scenario states that the model was highly accurate at launch and that precision degraded significantly over 18 months as fraudsters adapted. It does not describe the processor's exact tools, monitoring dashboards, or retraining schedule. A reasonable analysis is that the deployed model did not adapt quickly enough to changing fraud patterns, or that the existing monitoring and response process was insufficient.

A typical basic process is to train a model using historical transactions, test it on a held-out dataset, deploy it to score incoming payments, and periodically review outcomes. This can work initially, but a static model may become less useful when the operating environment changes.

3.2 Strengths of the Existing Model-Based Approach

- Automated scoring can evaluate a large number of transactions consistently.
- Historical data may reveal complex patterns that are difficult to express through manual rules alone.
- Scores can help analysts prioritize transactions for review.
- Once deployed, the model can support rapid decisions when the infrastructure is designed for low latency.

3.3 Limitations and Gaps

- Monitoring that relies only on occasional reviews may discover degradation too late.
- Historical training data may not represent current fraud behaviour.
- Input-distribution changes alone cannot confirm a decline in precision.
- Retraining without data checks and candidate validation may introduce new errors.
- Without versioning, the team may not know which data and code produced a model.
- Without rollback, a poor release may remain active while teams investigate the cause.

3.4 Gap Analysis

Area	Potential gap	Recommended response

Edit with WPS Office

Data quality	Unexpected values or schema changes may go unnoticed.	Automated validation and data-quality alerts.
Model quality	Precision may decline between manual reviews.	Monitor delayed-label metrics and business outcomes.
Retraining	Model updates may be reactive or ad hoc.	Define evidence-based triggers and a repeatable workflow.
Deployment	Candidate may affect users before it is sufficiently tested.	Use shadow deployment and staged rollout.
Governance	Decisions may lack traceability.	Record versions, evaluations, approvals, and incidents.

4. Propose a Suitable Software-Based Solution

4.1 Proposed MLOps Architecture

The proposed architecture creates a continuous feedback loop. Transaction data is validated before use; feature, prediction, and operational signals are monitored; alerts initiate investigation; verified data can be used to train a candidate; and a candidate is deployed only after required evaluation and release checks.

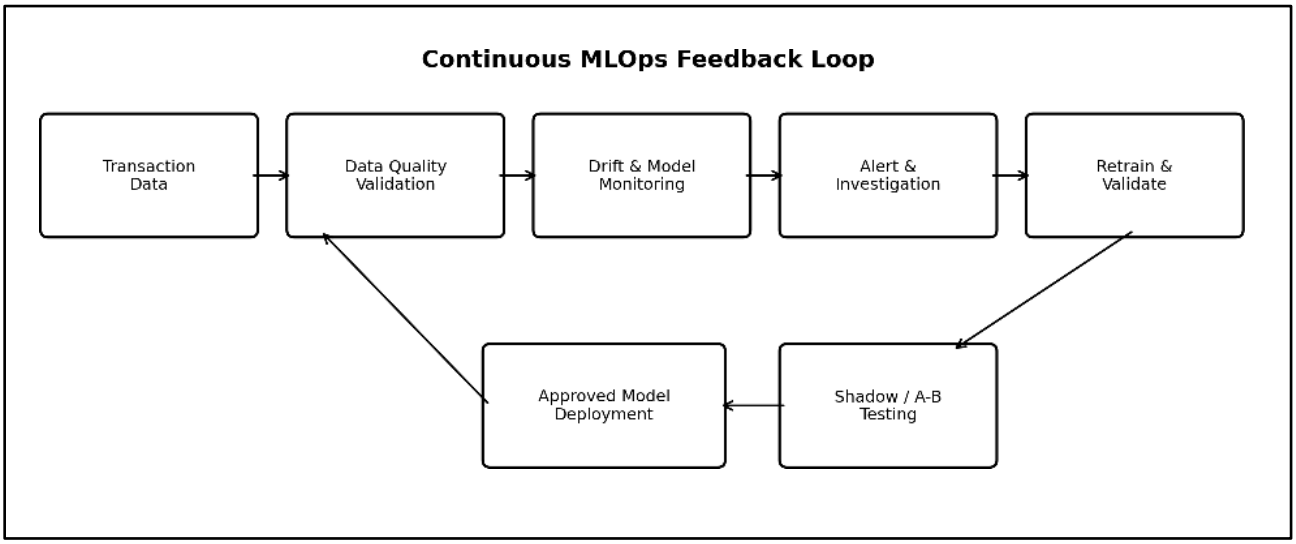


Fig 4 Continuous MLOps pipeline for fraud model monitoring and recovery

Edit with WPS Office

4.2 Major Functional Modules

Module 1: Data Ingestion and Validation

This module collects approved transaction features, timestamps, prediction scores, model version identifiers, and confirmed labels when they become available. It checks required fields, schema compatibility, duplicate records, valid ranges, and unexpected category values. Invalid data should be quarantined or flagged rather than silently passed to the model.

Module 2: Drift and Prediction Monitoring

This module compares current feature distributions with a trusted reference baseline. It also monitors changes in score distributions, decision rates, and manual-review volume. Statistical measures such as Population Stability Index (PSI) or suitable distribution tests may be used where appropriate. Baselines and thresholds should be documented and periodically reviewed.

Module 3: Model Performance Monitoring

When confirmed labels arrive, this module calculates precision, recall, false-positive rate, PR-AUC, and relevant business measures such as chargebacks or estimated fraud loss. Metrics should be assessed over time windows and, where appropriate, transaction segments. The system should display the sample size and label coverage so that users understand uncertainty.

4.3 Pipeline Modules (Continued)

Module 4: Alerting and Investigation

Alerts are generated when a quality check fails, drift persists beyond a threshold, performance falls below an approved limit, or service health deteriorates. An alert should identify the metric, time window, model version, affected data, severity, and suggested next step. Analysts should investigate whether the cause is data quality, seasonality, a genuine fraud shift, or an operational issue.

Module 5: Retraining and Model Registry

The retraining workflow selects recent, quality-checked labelled data and trains a candidate model. The registry stores the model artifact alongside its training data reference, code version, configuration, parameters, evaluation results, and approval status. This supports reproducibility and comparison. Candidate models should not be promoted simply because they are newer.

Module 6: Validation and Deployment

The candidate is compared with the current production model using a time-aware holdout set and

Edit with WPS Office

agreed acceptance criteria. If it passes, it enters shadow mode. After successful shadow evaluation, a controlled A/B test may be performed where suitable. Gradual rollout and rollback protect the payment service if unexpected behaviour occurs.

4.4 Monitoring Strategy

Monitoring layer	Signals to inspect	Response
Data quality	Missing values, schema changes, invalid ranges, duplicates.	Fix or quarantine problematic data.
Data drift	Feature distribution shifts and category changes.	Investigate source and business changes.
Prediction drift	Score distribution, flag rate, review volume.	Compare with baseline and examine segments.
Outcome performance	Precision, recall, false positives, PR-AUC, fraud loss.	Review model quality when labels are available.
Operational health	Latency, throughput, errors, pipeline failures.	Restore service and resolve infrastructure issues.

The monitoring dashboard should clearly distinguish leading indicators, such as data drift, from lagging indicators, such as confirmed fraud precision. Alerts should be prioritized by severity to avoid overwhelming the response team.

4.5 Retraining Triggers and Model Validation

Retraining Triggers

Retraining should be considered when one or more agreed conditions persist. Examples include precision below the approved minimum, recall deterioration, an increase in fraud loss, sustained drift in important features, a major change in transaction channels, or a scheduled review date. Thresholds should be based on business risk and historical variation; arbitrary thresholds can create unnecessary retraining.

- Check data quality and label coverage before starting a training run.
- Use time-aware train, validation, and test splits to reduce leakage.
- Compare the candidate against the current model and relevant baseline rules.

Edit with WPS Office

- Review precision, recall, false-positive rate, PR-AUC, latency, and segment-level results.
- Require approval when risk thresholds or business policies may be affected.
- Store results so the team can reproduce the decision later.

4.6 Shadow Deployment

Shadow deployment sends the same incoming transactions to the current model and a candidate model. The current model continues to control production decisions, while the candidate's predictions are recorded for comparison. This approach tests the candidate under real traffic without allowing its predictions to directly affect customers.

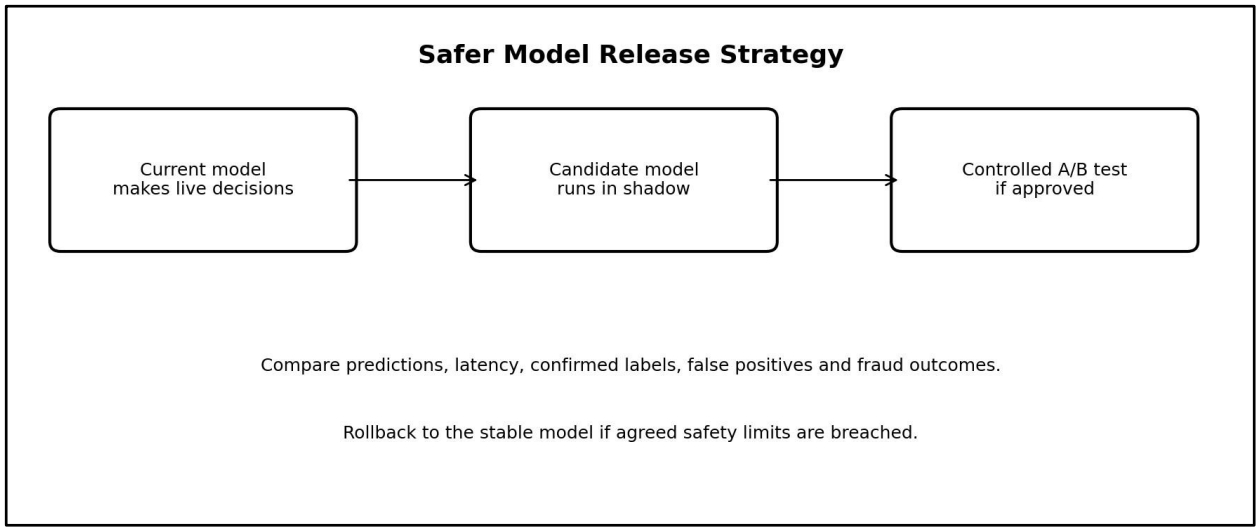


Fig 4.5 Safer model release strategy: shadow evaluation before controlled A/B test

4.7 A/B Testing

After the candidate passes shadow evaluation, an A/B test can compare the current model with the candidate on a controlled share of eligible traffic. The assignment method and success criteria should be set in advance. Teams should compare outcomes fairly and account for delayed labels, sample size, and differences in traffic composition. The experiment must have clear stop conditions and a documented rollback plan.

Edit with WPS Office

4.8 Functional and Non-Functional Requirements

Functional requirements	Non-functional requirements
Validate transaction data and log data-quality failures.	Reliable operation with recoverable failures.
Calculate drift indicators and create alerts.	Scalable processing as transaction volume grows.
Track model performance after labels arrive.	Latency within agreed payment-service limits.
Train, evaluate, and register model versions.	Maintainable and testable pipeline components.
Support shadow tests, A/B tests, and rollback.	Encryption, access control, and data minimization.
Record approvals, incidents, and deployment history.	Auditable, reproducible decisions and experiments.

4.9 Software Engineering Principles

Modularity: Separate ingestion, validation, monitoring, training, registry, deployment, and alerting into components with clear interfaces.

Scalability: Scale scoring and monitoring workloads independently as transaction volumes grow.

Maintainability: Use version control, automated tests, reusable pipeline steps, documentation, and configuration management.

Reliability: Use health checks, retries, alerting, staged releases, and rollback to a known stable model.

Usability: Provide dashboards with clear metric trends, thresholds, model versions, and actionable alert details.

Security: Apply least-privilege access, encryption, secrets management, audit logs, and limited retention of sensitive data.

4.10 Security, Privacy, and Governance

Financial transaction data should be accessible only to authorized roles. Data should be encrypted during transmission and storage, secrets should not be placed in source code, and logs should avoid exposing unnecessary personal details. Model releases should have named owners, documented acceptance checks, and a history of approvals. These controls improve trust and make incidents easier to investigate.

Edit with WPS Office

5. Justify the Chosen Methodology / Framework

5.1 Agile with MLOps and DevOps Practices

Agile is suitable because fraud patterns, data characteristics, and business requirements change over time. The project can be developed in short iterations, with stakeholder feedback at the end of each sprint. MLOps and DevOps practices complement Agile by automating testing, versioning, packaging, deployment, monitoring, and rollback.

Iteration	Deliverable	Acceptance check
Sprint 1	Data validation and baseline metrics.	Known data errors are detected and logged.
Sprint 2	Drift and performance dashboard.	Alerts identify metric, time window, and model version.
Sprint 3	Repeatable retraining and model registry.	Candidate can be reproduced and compared.
Sprint 4	Shadow evaluation and controlled rollout.	Safety thresholds and rollback are demonstrated.

5.2 How the Methodology Supports Success

- Sprint planning prioritizes the most important monitoring and safety requirements.
- Continuous integration runs code checks, automated tests, and data validation.
- Model evaluation gates prevent untested candidates from being promoted.
- Sprint reviews gather feedback from fraud analysts, engineers, and business teams.
- Retrospectives identify pipeline failures and improve the process.
- Versioned releases support reproducibility and rollback.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

- Earlier warning when transaction patterns or model performance begin to change.
- More evidence-based decisions about when to retrain.
- Safer model releases through shadow testing, A/B testing, and rollback.
- Clearer accountability through dashboards, model history, and audit records.

Edit with WPS Office

- Improved balance between fraud reduction and legitimate-customer experience.

6.2 Risks and Limitations

- Confirmed fraud labels may arrive late, delaying reliable performance evaluation.
- Drift metrics may generate false alarms or miss changes that affect outcomes.
- A candidate can perform well on historical tests yet fail against future fraud tactics.
- A/B tests require enough representative data and fair assignment.
- Continuous MLOps needs infrastructure, specialist skills, monitoring, and maintenance.

6.3 Future Enhancements

Future enhancements may include segment-specific drift monitoring, analyst feedback loops, champion–challenger model management, explainability for individual decisions, privacy-preserving analytics, and automatic rollback when predefined safety limits are crossed. Thresholds should be reviewed periodically as transaction patterns and business priorities evolve.

Conclusion

The case study demonstrates that a fraud detection model can lose precision when fraudsters adapt. A continuous MLOps pipeline can address this risk through data-quality checks, drift and performance monitoring, evidence-based retraining, shadow deployment, controlled A/B testing, and rollback. Agile development supported by MLOps/DevOps enables incremental delivery and ongoing maintenance. Reliable labels, human oversight, security, and measurable release criteria remain essential.

Reference Note

The scenario and assessment structure are based on the provided CSA10 CO1 Assessment Tool 1 case study. The technical pipeline, metric explanations, sample sprint plan, and figures are proposed explanatory content. The precision-degradation chart is illustrative only and should not be interpreted as actual measured data from the payment processor.

Edit with WPS Office